Flight Delay Prediction

36-673 Final Project

Ellie Huang (elliehua) and Nikita Jayaprakash (njayapra)

2026-04-22

Data Prep

Load Libraries and Data

```
# Load data
flights2023 <- read_csv(here("data", "flights2023.csv")) |>
 clean_names()

flights2024 <- read_csv(here("data", "flights2024.csv")) |>
 clean_names()

test <- read_csv(here("data", "flights2025.csv")) |>
 clean_names()

test_visible <- read_csv(here("data", "flights2025_visible.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

test_guess <- read_csv(here("data", "flights2025_guess.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

# Build training set
train <- bind_rows(
 flights2023 |> mutate(data_year = 2023),
 flights2024 |> mutate(data_year = 2024)
)
```

Clean and Filter Data

```
# Apply the same cleaning steps to each data frame so train and test sets are consistent
clean_flights <- function(df) {
 df |>
 clean_names() |>
 mutate(
 fl_date = as.Date(fl_date),
 dep_del15 = as.numeric(dep_del15),
 op_unique_carrier = as.factor(op_unique_carrier),
 origin = as.factor(origin),
 dest = as.factor(dest),
```

```r
 sched_dep_hour = floor(crs_dep_time / 100),
 month = as.factor(month),
 day_of_week = as.factor(day_of_week)
 )
}

train <- clean_flights(train)
test <- clean_flights(test)
test_visible <- clean_flights(test_visible)
test_guess <- clean_flights(test_guess)
```

```r
# Restrict to departures from PIT
train <- train |>
 filter(origin == "PIT")

test_visible <- test_visible |>
 filter(origin == "PIT")

test_guess <- test_guess |>
 filter(origin == "PIT")

# Quick check of the target distribution
table(train$dep_del15, useNA = "ifany")
```

```
##
##     0     1  <NA>
## 59534 10972   950
```

We focus on Pittsburgh departures because that is the airport represented in the prediction task. We also use `sched_dep_hour` rather than actual departure time, since actual departure time is not available for the flights we need to predict.

Exploratory Data Analysis

The goal of the EDA is not just to describe the data, but to identify patterns that are both predictive and available before departure. In particular, we want to see whether delay risk depends in a stable way on scheduling, carrier, route, seasonality, and same-day operating conditions.

Missingness

```r
# Missingness in the main timing and target fields
colSums(is.na(train[, c("dep_del15", "dep_time", "crs_dep_time")]))
```

```
##    dep_del15     dep_time crs_dep_time
##          950          950            0
```

```r
# Variables with nonzero missingness
train |>
 summarise(across(everything(), ~mean(is.na(.)))) |>
 pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") |>
 filter(pct_missing > 0) |>
 arrange(desc(pct_missing)) |>
 print(n = 30)
```

```
## # A tibble: 27 x 2
```

```
##    variable            pct_missing
##    <chr>                     <dbl>
##  1 first_dep_time          0.995
##  2 total_add_gtime         0.995
##  3 longest_add_gtime       0.995
##  4 cancellation_code       0.986
##  5 carrier_delay           0.837
##  6 weather_delay           0.837
##  7 nas_delay               0.837
##  8 security_delay          0.837
##  9 late_aircraft_delay     0.837
## 10 fl_date                 0.609
## 11 arr_delay               0.0154
## 12 arr_delay_new           0.0154
## 13 arr_del15               0.0154
## 14 arr_delay_group         0.0154
## 15 actual_elapsed_time     0.0154
## 16 air_time                0.0154
## 17 wheels_on               0.0138
## 18 taxi_in                 0.0138
## 19 arr_time                0.0138
## 20 taxi_out                0.0137
## 21 wheels_off              0.0137
## 22 dep_time                0.0133
## 23 dep_delay               0.0133
## 24 dep_delay_new           0.0133
## 25 dep_del15               0.0133
## 26 dep_delay_group         0.0133
## 27 tail_num                0.00158
```

```r
# Missingness among key pre-departure predictors
key_predictors <- c(
 "dep_del15", "crs_dep_time", "sched_dep_hour", "op_unique_carrier",
 "dest", "distance", "month", "day_of_week", "flights"
)

key_predictors <- intersect(key_predictors, names(train))

train |>
 summarise(across(all_of(key_predictors), ~mean(is.na(.)))) |>
 pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") |>
 mutate(pct_missing = percent(pct_missing, accuracy = 0.1))
```

```
## # A tibble: 9 x 2
##   variable          pct_missing
##   <chr>             <chr>
## 1 dep_del15         1.3%
## 2 crs_dep_time      0.0%
## 3 sched_dep_hour    0.0%
## 4 op_unique_carrier 0.0%
## 5 dest              0.0%
## 6 distance          0.0%
## 7 month             0.0%
## 8 day_of_week       0.0%
## 9 flights           0.0%
```

```r
# Check whether missing targets line up with likely cancellations
train |>
 mutate(cancelled_proxy = is.na(dep_time) & is.na(dep_del15)) |>
 count(cancelled_proxy, missing_target = is.na(dep_del15))
```

```
## # A tibble: 2 x 3
##   cancelled_proxy missing_target     n
##   <lgl>           <lgl>          <int>
## 1 FALSE           FALSE          70506
## 2 TRUE            TRUE             950
```

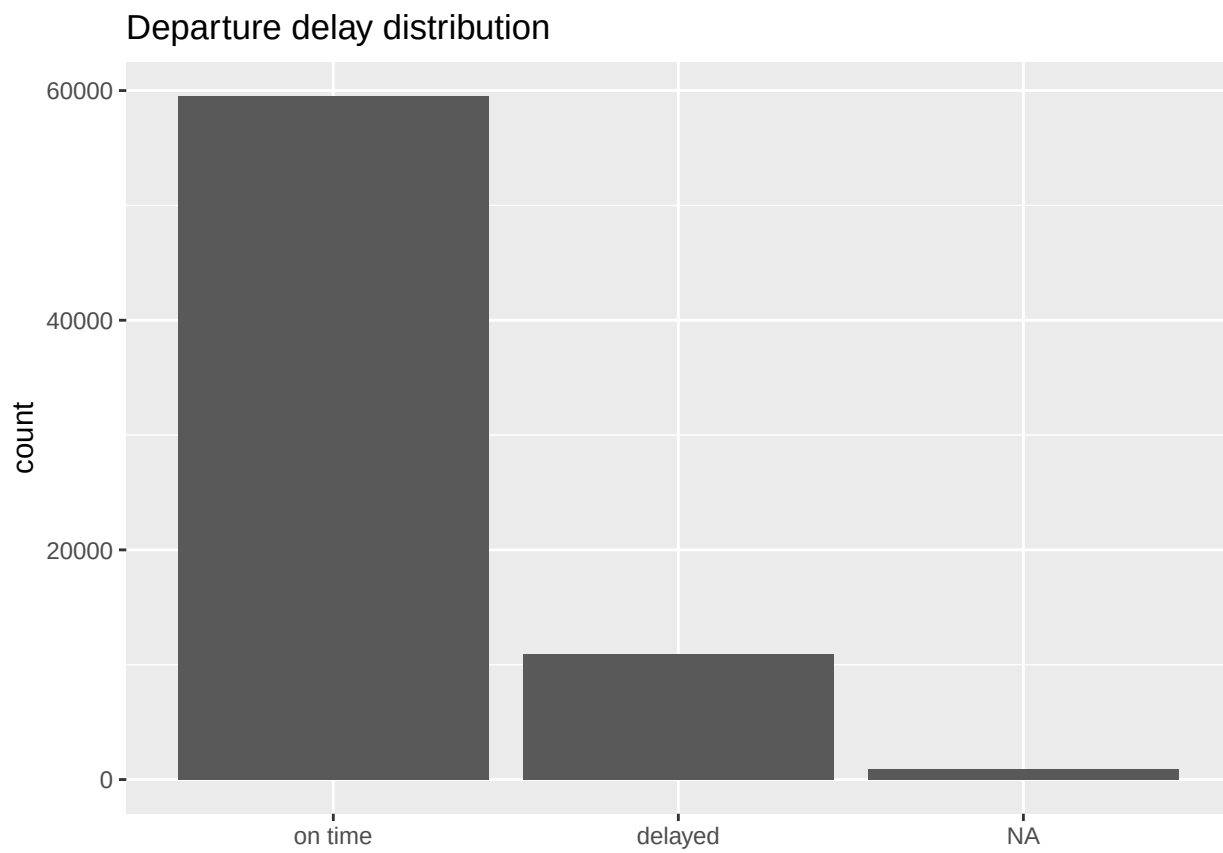
Missing target values align exactly with likely cancellations, so this is not unexplained missingness. We therefore treat these rows as cancelled flights and exclude them from the delay prediction task.

Departure Delay Distribution

```r
# Target distribution
train |>
 count(dep_del15)
```

```
## # A tibble: 3 x 2
##   dep_del15     n
##       <dbl> <int>
## 1         0 59534
## 2         1 10972
## 3        NA   950
```

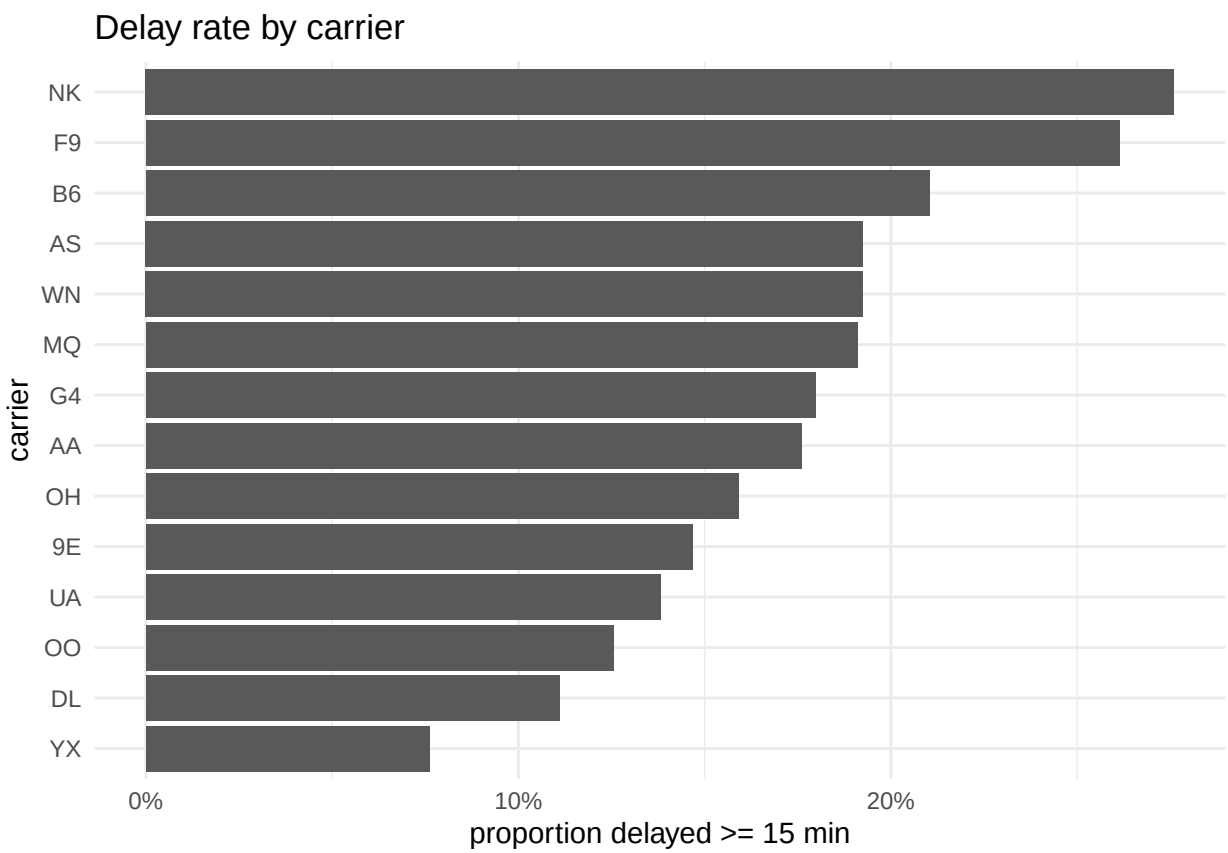
```r
train |>
 ggplot(aes(x = factor(dep_del15, labels = c("on time", "delayed")))) +
 geom_bar() +
 labs(
 title = "Departure delay distribution",
 x = NULL,
 y = "count"
 )
```



The classes are roughly 80/20 on-time versus delayed. This imbalance is worth keeping in mind, but it is not too concerning since our main evaluation metric is AUC rather than raw accuracy.

Delay Rate by Carrier

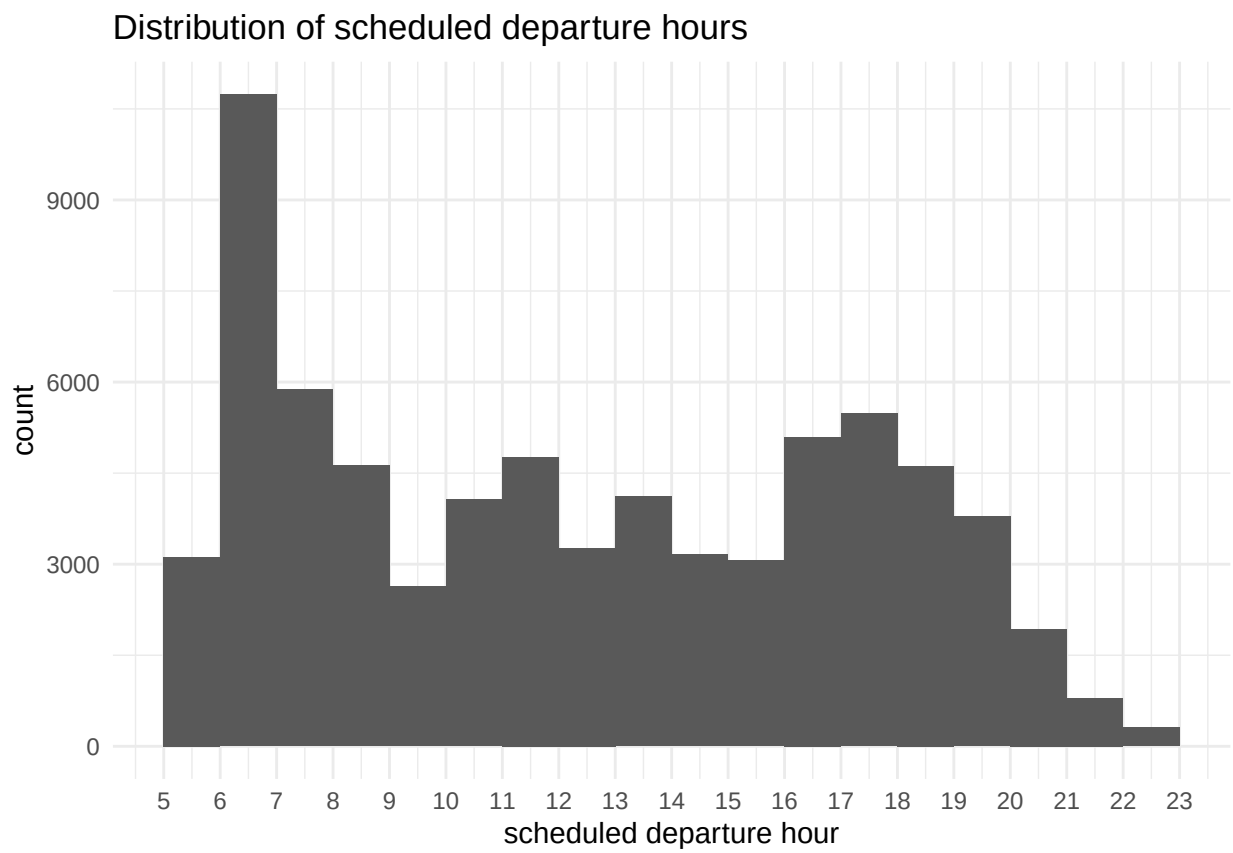
```
carrier_delay_tbl <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(op_unique_carrier) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 arrange(desc(delay_rate))

carrier_delay_tbl |>
 ggplot(aes(x = reorder(op_unique_carrier, delay_rate), y = delay_rate)) +
 geom_col() +
 coord_flip() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by carrier",
 x = "carrier",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```



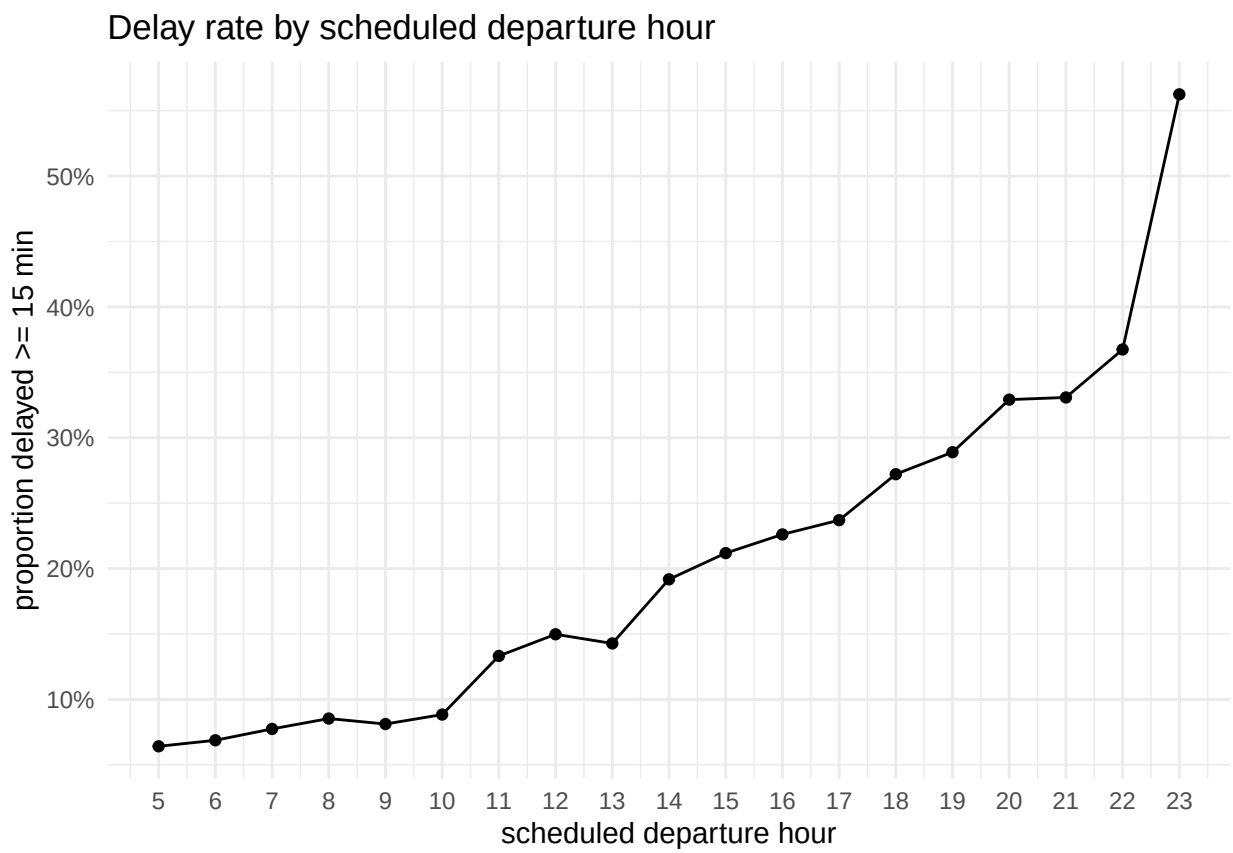
Delay rates vary meaningfully across carriers, so carrier should be a useful predictor. At the same time, some carriers have much lower flight counts at PIT than others, so raw group means will be noisy. That motivates using smoothed historical delay rates later in feature engineering instead of unsmoothed averages.

Delay Rate by Scheduled Departure Hour

```
# Distribution of scheduled departure times
train |>
 ggplot(aes(x = sched_dep_hour)) +
 geom_histogram(binwidth = 1, boundary = 0, closed = "left") +
 scale_x_continuous(breaks = 0:23) +
 labs(
 title = "Distribution of scheduled departure hours",
 x = "scheduled departure hour",
 y = "count"
 ) +
 theme_minimal()
```




```
hour_delay_tbl <- train |>
 filter(!is.na(dep_del15), !is.na(sched_dep_hour)) |>
 group_by(sched_dep_hour) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 )

hour_delay_tbl |>
 ggplot(aes(x = sched_dep_hour, y = delay_rate)) +
 geom_line() +
 geom_point() +
 scale_x_continuous(breaks = 0:23) +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by scheduled departure hour",
 x = "scheduled departure hour",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```

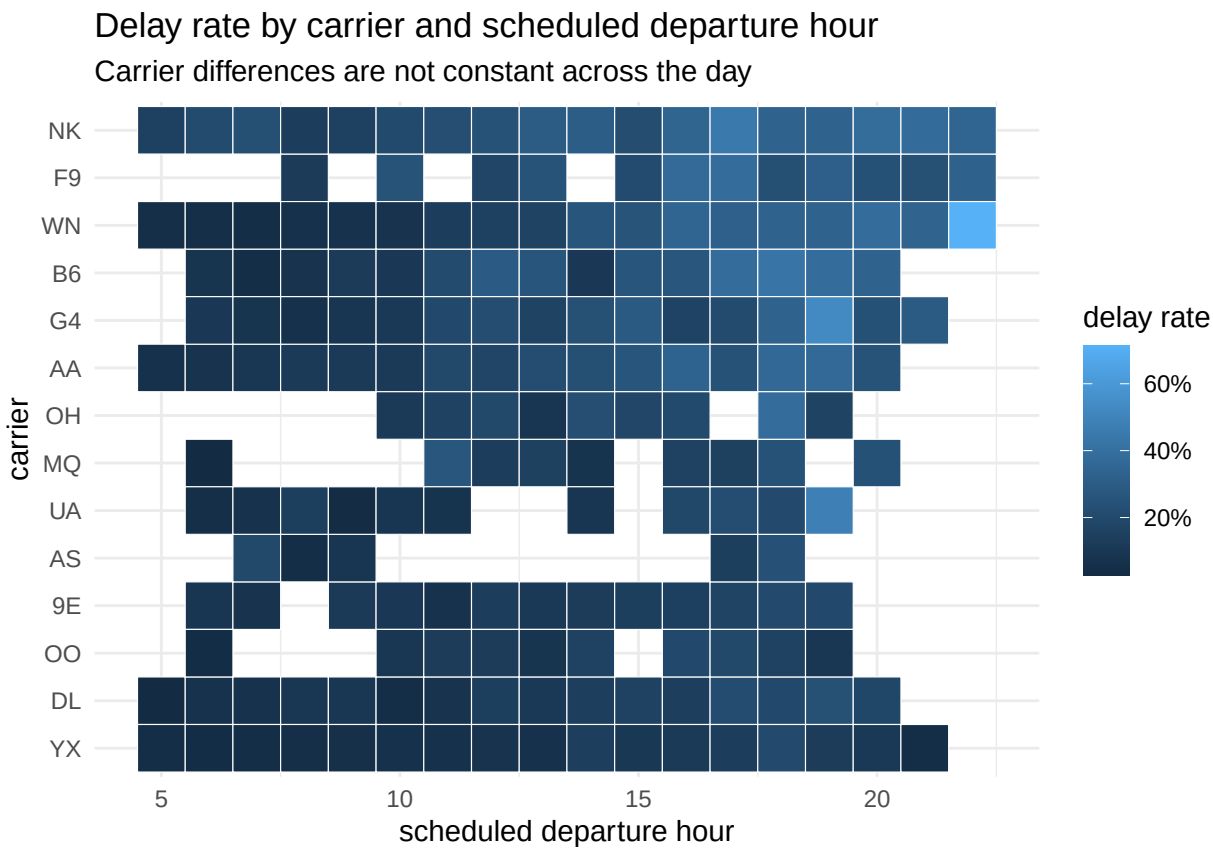


Delay rates rise steadily over the course of the day and are highest in the evening. This fits the idea of cascading delays: early flights start the day with fewer disruptions from previous legs, while later flights are more exposed to effects from earlier problems. This makes scheduled departure hour one of the strongest single predictors in the dataset.

Carrier-by-Hour Interaction

```
carrier_hour_tbl <- train |>
 filter(!is.na(dep_del15), !is.na(sched_dep_hour)) |>
 group_by(op_unique_carrier, sched_dep_hour) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 filter(n >= 20)

carrier_hour_tbl |>
 ggplot(
 aes(
 x = sched_dep_hour,
 y = fct_reorder(op_unique_carrier, delay_rate, .fun = mean),
 fill = delay_rate
 )
 ) +
 geom_tile(color = "white") +
```

```
scale_fill_gradient(labels = percent_format(accuracy = 1)) +
labs(
title = "Delay rate by carrier and scheduled departure hour",
subtitle = "Carrier differences are not constant across the day",
x = "scheduled departure hour",
y = "carrier",
fill = "delay rate"
) +
theme_minimal()
```

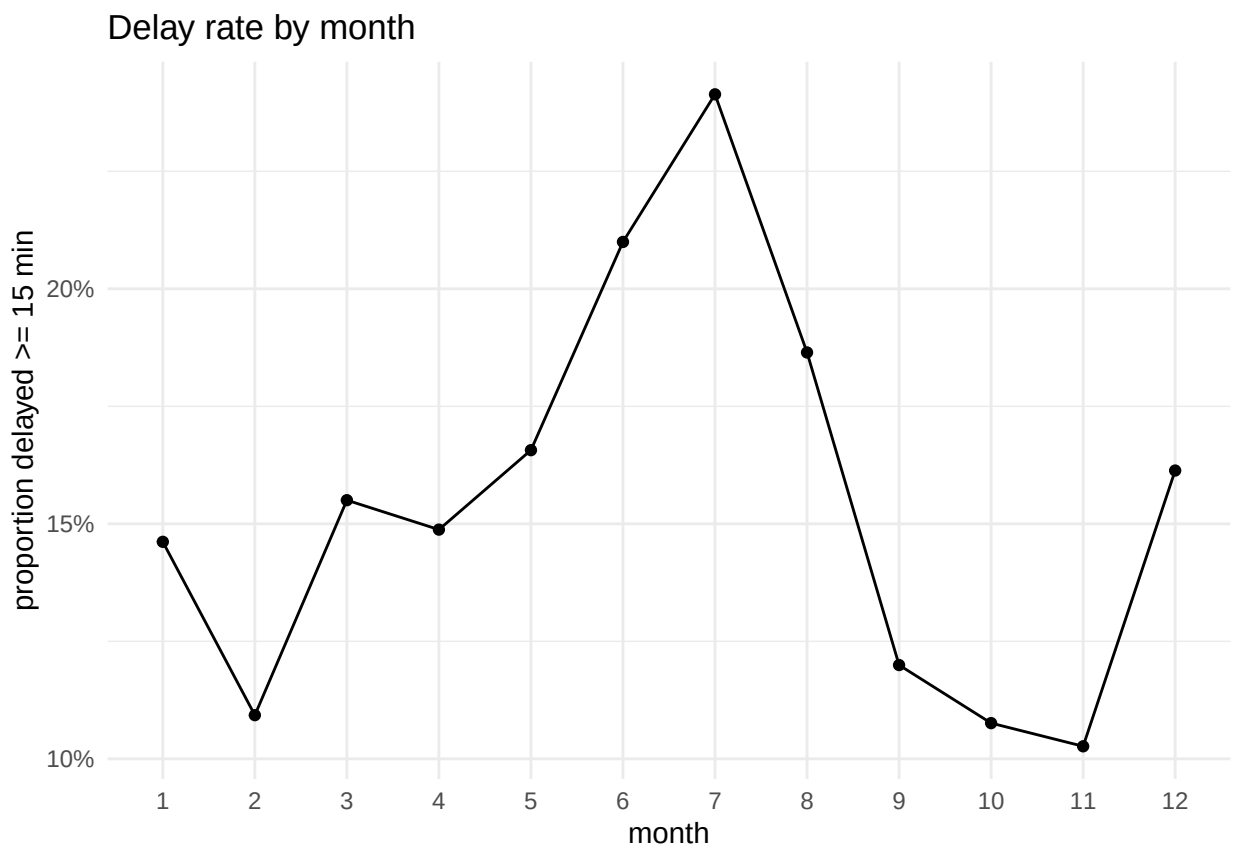


The carrier effect is not the same at every hour. Some carriers remain fairly stable across the day, while others become much more delay-prone in the evening. This suggests that a model with only main effects may miss meaningful structure, so either an explicit carrier-by-hour feature or a nonlinear model should help.

Seasonality and Calendar Effects

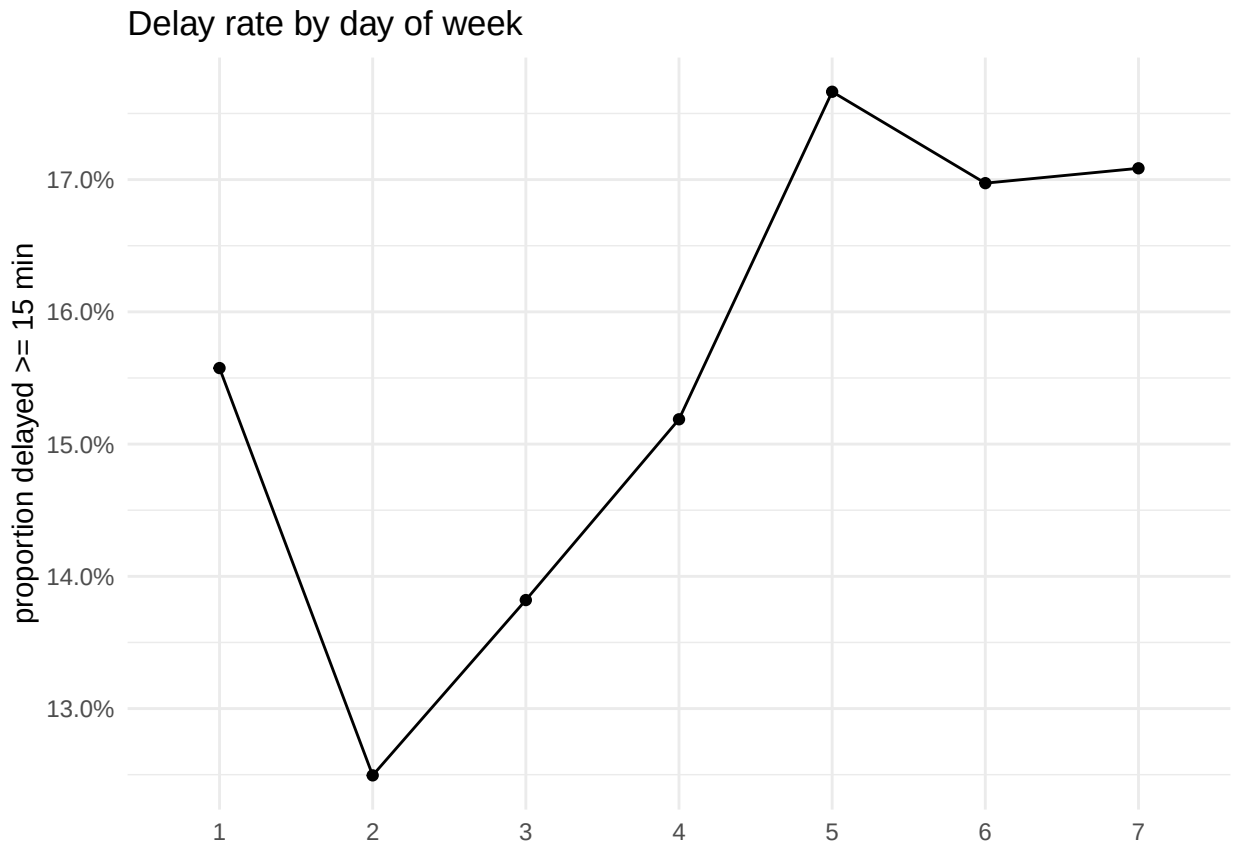
```
# Delay rate by month
train |>
 filter(!is.na(dep_del15)) |>
 group_by(month) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
 ) |>
 ggplot(aes(x = month, y = delay_rate, group = 1)) +
 geom_line() +
```

```
 geom_point() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by month",
 x = "month",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```

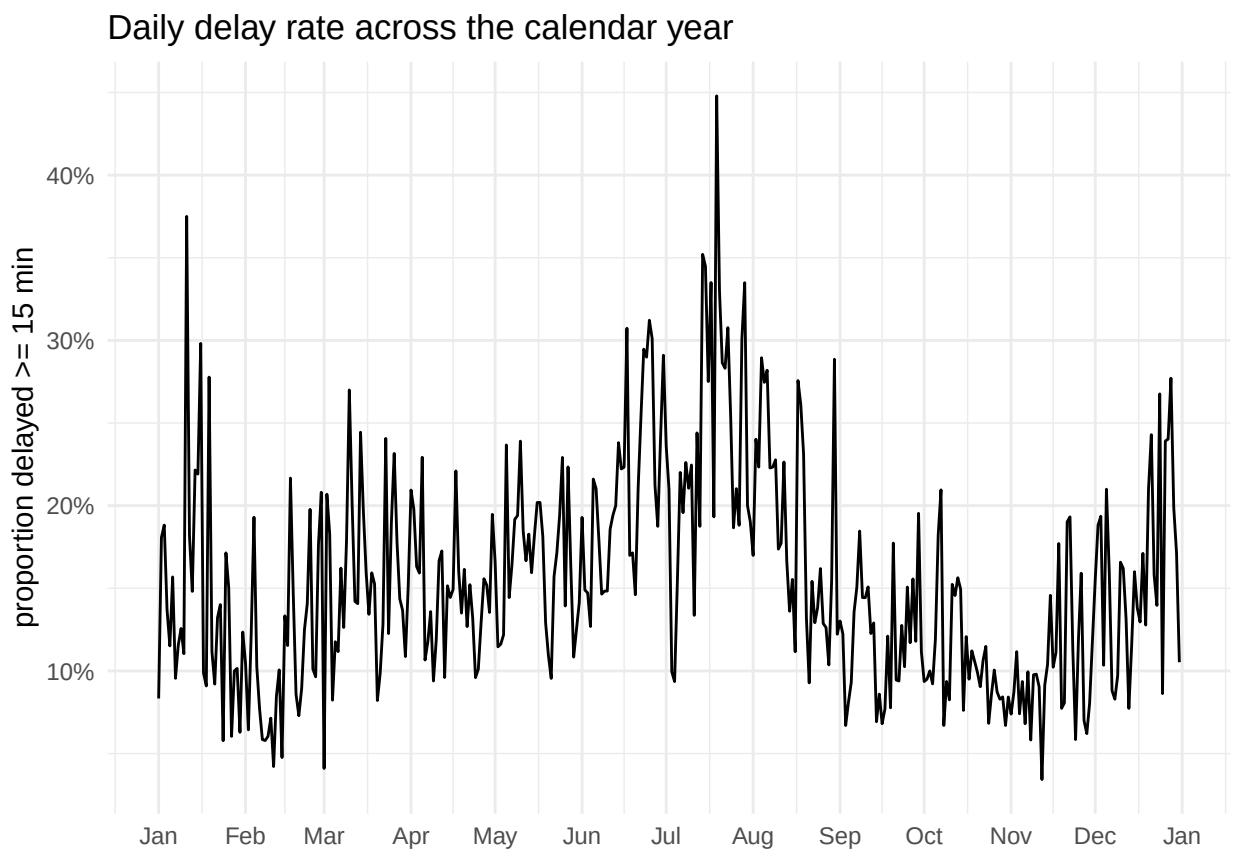



```
# Delay rate by day of week
train |>
 filter(!is.na(dep_del15)) |>
 group_by(day_of_week) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
 ) |>
 ggplot(aes(x = day_of_week, y = delay_rate, group = 1)) +
 geom_line() +
 geom_point() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by day of week",
 x = NULL,
 y = "proportion delayed >= 15 min"
 ) +
```

```
theme_minimal()
```




```
# Delay rate by calendar date, aligned to a common year for display
train |>
 filter(!is.na(dep_del15)) |>
 mutate(plot_date = make_date(year = 2010, month = as.integer(as.character(month)), day = day_of_month)
 group_by(plot_date) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
 ) |>
 ggplot(aes(x = plot_date, y = delay_rate)) +
 geom_line() +
 scale_x_date(date_labels = "%b", date_breaks = "1 month") +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Daily delay rate across the calendar year",
 x = NULL,
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```

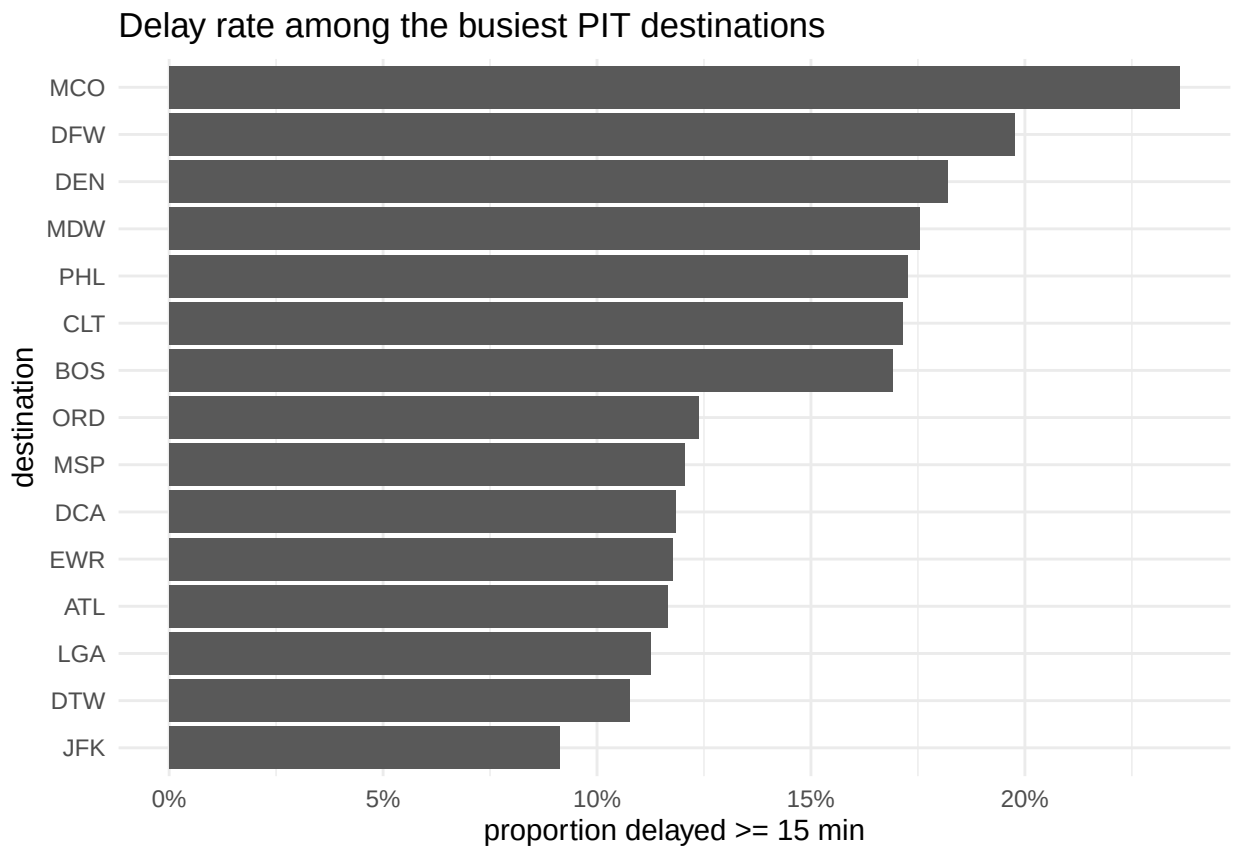


Delays are worse in the summer months and in December, which is consistent with heavier travel volume and more weather-related disruption. The day-of-week pattern is milder, but it still suggests that weekly operating patterns contain some signal. The daily calendar plot also shows that these seasonal peaks are not spread evenly across the month, which supports adding a simple calendar feature rather than relying only on month.

Destination and Distance Effects

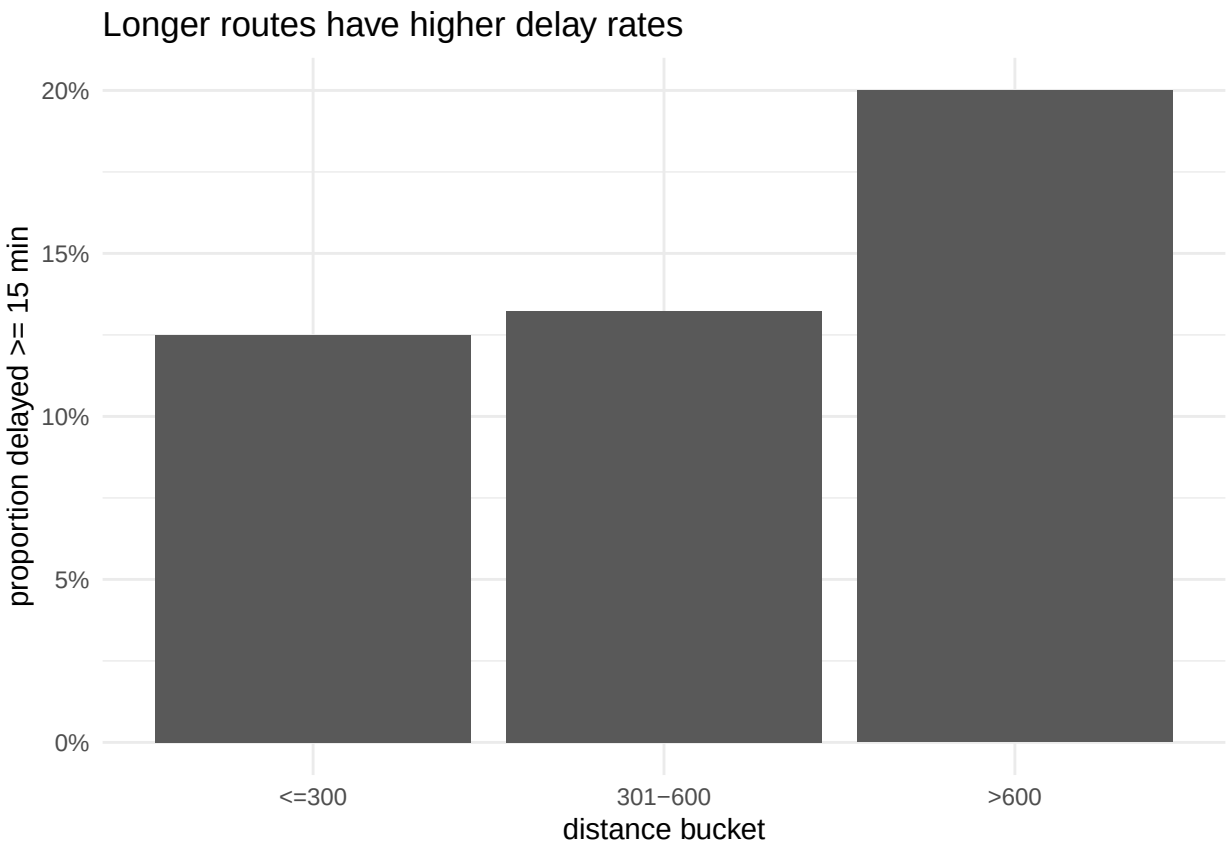
```
top_dests <- train |>
 count(dest, sort = TRUE) |>
 slice_head(n = 15) |>
 pull(dest)

train |>
 filter(!is.na(dep_del15), dest %in% top_dests) |>
 group_by(dest) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 arrange(delay_rate) |>
 ggplot(aes(x = reorder(dest, delay_rate), y = delay_rate)) +
 geom_col() +
 coord_flip() +
 scale_y_continuous(labels = percent_format()) +
```

```
 labs(
 title = "Delay rate among the busiest PIT destinations",
 x = "destination",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```




```
train <- train |>
 mutate(
 distance_bucket = case_when(
 distance > 600 ~ ">600",
 distance > 300 ~ "301-600",
 TRUE ~ "<=300"
 ),
 distance_bucket = factor(
 distance_bucket,
 levels = c("<=300", "301-600", ">600")
 )
 )

train |>
 filter(!is.na(dep_del15), !is.na(distance_bucket)) |>
 group_by(distance_bucket) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
```

```
 .groups = "drop"
 ) |>
 ggplot(aes(x = distance_bucket, y = delay_rate)) +
 geom_col() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Longer routes have higher delay rates",
 x = "distance bucket",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```

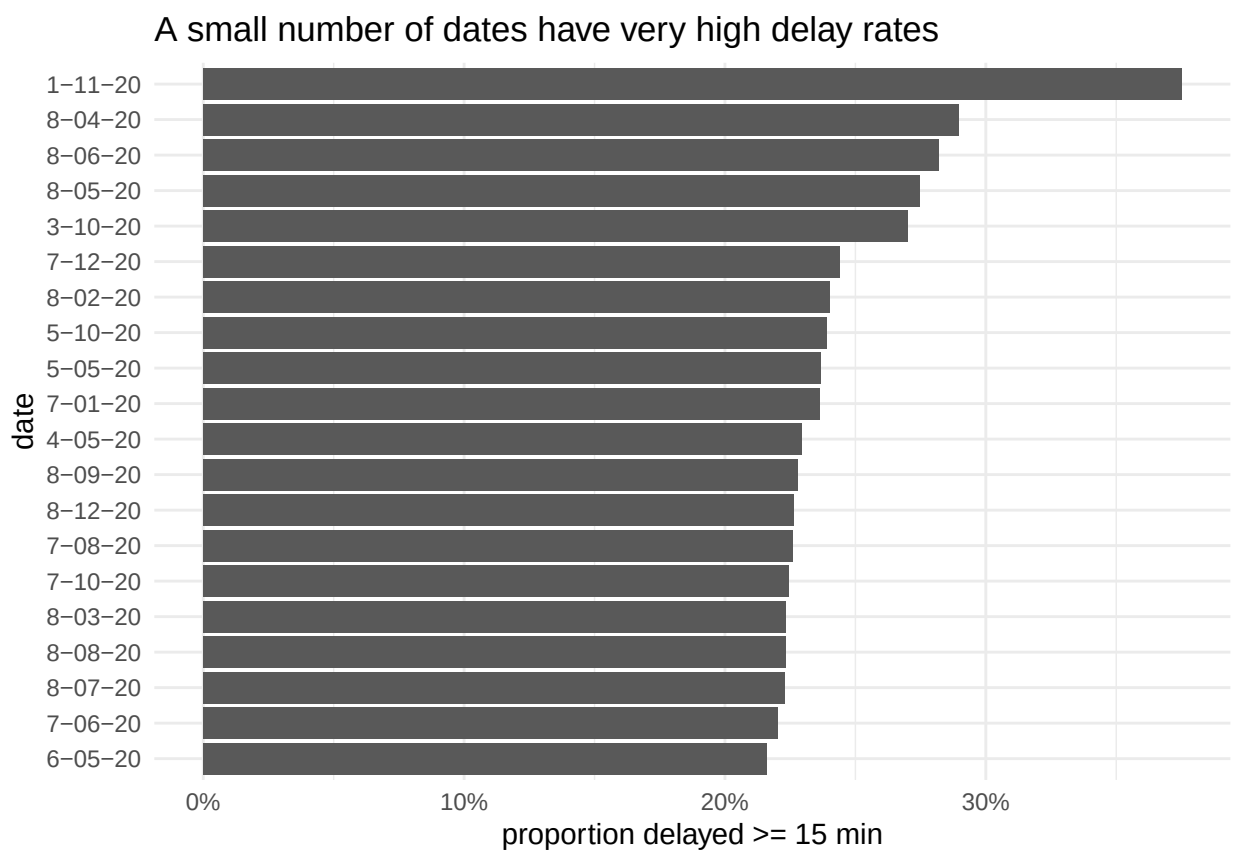


Delay rates differ across destinations, even among the busiest PIT routes, which suggests that route context adds information beyond carrier and time of day alone. Longer routes also show higher delay rates on average, so distance should remain in the model and may be easier to capture through both a raw continuous version and a bucketed factor version

High-Delay Days

```
bad_days_tbl <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(fl_date) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
```

```r
  ) |>
  filter(n >= 10) |>
  arrange(desc(delay_rate))

bad_days_tbl |>
  slice_head(n = 20) |>
  ggplot(aes(x = reorder(as.character(fl_date), delay_rate), y = delay_rate)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(labels = percent_format()) +
  labs(
  title = "A small number of dates have very high delay rates",
  x = "date",
  y = "proportion delayed >= 15 min"
  ) +
  theme_minimal()
```



A small number of days have unusually high delay rates, which suggests that there are broader operating shocks affecting many flights at once. That motivates adding dynamic same-day features based on earlier visible flights, along with a historical calendar-day disruption feature learned from the training data.

Feature Engineering

The EDA suggests that delay risk depends on a mix of stable historical tendencies and day-specific operating conditions. Based on those patterns, we engineer features that capture time-of-day effects, carrier and route

history, route distance, seasonality, and same-day disruption.

Add Basic Modeling Features

```
add_basic_features <- function(df) {
 df |>
 mutate(
 peak_season = if_else(as.integer(as.character(month)) %in% c(6, 7, 8, 12), 1, 0),
 is_evening = if_else(sched_dep_hour >= 18, 1, 0),
 is_morning = if_else(sched_dep_hour <= 8, 1, 0),
 distance_bucket = case_when(
 distance > 600 ~ ">600",
 distance > 300 ~ "301-600",
 TRUE ~ "<=300"
 ),
 carrier_hour = paste(op_unique_carrier, sched_dep_hour, sep = "_")
 )
}

train <- add_basic_features(train)
test_visible <- add_basic_features(test_visible)
test_guess <- add_basic_features(test_guess)
```

These features directly reflect the strongest patterns from the EDA. In particular, `peak_season`, the morning and evening indicators, the distance bucket, and the carrier-by-hour combination all transform observed structure in the data into modeling variables.

Build Smoothed Historical Delay-Rate Features

```
global_delay_rate <- mean(train$dep_del15, na.rm = TRUE)

make_smoothed_rates <- function(df, group_var, target = "dep_del15", m = 100) {
 group_sym <- rlang::sym(group_var)
 target_sym <- rlang::sym(target)

 df |>
 filter(!is.na(!!target_sym)) |>
 group_by(!!group_sym) |>
 summarise(
 n = n(),
 mean_target = mean(!!target_sym),
 .groups = "drop"
 ) |>
 mutate(
 smoothed_rate = (n * mean_target + m * global_delay_rate) / (n + m)
 ) |>
 select(!!group_sym, smoothed_rate, n)
}
```

```
carrier_rates <- make_smoothed_rates(train, "op_unique_carrier", m = 200) |>
 rename(
 carrier_delay_rate_smooth = smoothed_rate,
 carrier_n = n
 )
```

```r
dest_rates <- make_smoothed_rates(train, "dest", m = 200) |>
 rename(
 dest_delay_rate_smooth = smoothed_rate,
 dest_n = n
 )

carrier_hour_rates <- make_smoothed_rates(train, "carrier_hour", m = 100) |>
 rename(
 carrier_hour_delay_rate_smooth = smoothed_rate,
 carrier_hour_n = n
 )

flight_num_rates <- make_smoothed_rates(train, "flights", m = 150) |>
 rename(
 flight_num_delay_rate_smooth = smoothed_rate,
 flight_num_n = n
 )
```

Rather than using raw historical means, we shrink each group toward the global average when the number of observations is small. This makes the features more stable, especially for carriers, routes, or flight numbers that appear only a limited number of times at PIT.

Apply Historical Features to Train and Test

```r
apply_rate_features <- function(df) {
 df |>
 left_join(carrier_rates, by = "op_unique_carrier") |>
 left_join(dest_rates, by = "dest") |>
 left_join(carrier_hour_rates, by = "carrier_hour") |>
 left_join(flight_num_rates, by = "flights") |>
 mutate(
 carrier_delay_rate_smooth = coalesce(carrier_delay_rate_smooth, global_delay_rate),
 dest_delay_rate_smooth = coalesce(dest_delay_rate_smooth, global_delay_rate),
 carrier_hour_delay_rate_smooth = coalesce(carrier_hour_delay_rate_smooth, global_delay_rate),
 flight_num_delay_rate_smooth = coalesce(flight_num_delay_rate_smooth, global_delay_rate)
 )
}

train_fe <- apply_rate_features(train)
test_visible_fe <- apply_rate_features(test_visible)
test_guess_fe <- apply_rate_features(test_guess)
```

All of these encodings are learned using the training data and then applied to the test sets. If a group in the test set was not seen in training, we fall back to the global delay rate.

Add Running Same-Day Delay Features

```r
add_running_day_features <- function(df) {
 df |>
 arrange(fl_date, crs_dep_time) |>
 group_by(fl_date) |>
 mutate(
 before_today = row_number() - 1,
```

```
 delays_before_today = lag(cumsum(replace_na(dep_del15, 0)), default = 0),
 prior_same_day_delay_rate = if_else(
 before_today > 0,
 delays_before_today / before_today,
 0
 )
 ) |>
 ungroup()
}

train_fe <- add_running_day_features(train_fe)
test_visible_fe <- add_running_day_features(test_visible_fe)
test_guess_fe <- add_running_day_features(test_guess_fe)
```

This feature captures how disrupted the system already is earlier in the day. It is especially useful because the prediction setup gives visibility into earlier flights on the same date.

```
test_visible_day_summary <- test_visible_fe |>
 filter(!is.na(dep_del15)) |>
 group_by(fl_date) |>
 summarise(
 visible_count = n(),
 visible_delay_rate = mean(dep_del15),
 .groups = "drop"
 )

test_guess_fe <- test_guess_fe |>
 left_join(test_visible_day_summary, by = "fl_date") |>
 mutate(
 visible_count = coalesce(visible_count, 0),
 visible_delay_rate = coalesce(visible_delay_rate, global_delay_rate)
 )
```

For the guess set, we cannot directly use the unknown future labels. Instead, we summarize the flights that are already visible on that date and merge those day-level summaries into the flights we need to predict.

Add Historical High-Disruption Calendar Signal

```
bad_day_lookup <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(month, day_of_month) |>
 summarise(
 avg_delay_rate_for_calendar_day = mean(dep_del15),
 .groups = "drop"
 ) |>
 mutate(
 high_disruption_calendar_day = if_else(
 avg_delay_rate_for_calendar_day >= quantile(avg_delay_rate_for_calendar_day, 0.90),
 1,
 0
 )
 )

train_fe <- train_fe |>
```

```r
 left_join(bad_day_lookup, by = c("month", "day_of_month"))

test_visible_fe <- test_visible_fe |>
 left_join(bad_day_lookup, by = c("month", "day_of_month"))

test_guess_fe <- test_guess_fe |>
 left_join(bad_day_lookup, by = c("month", "day_of_month"))
```

This feature is meant to capture recurring calendar periods that tend to have higher disruption, without adding a large number of very specific date indicators to the model.

Final Modeling Data Frames

```r
model_vars <- c(
 "dep_del15",
 "sched_dep_hour",
 #"op_unique_carrier",
 #"dest",
 "distance",
 "distance_bucket",
 "month",
 "day_of_week",
 "peak_season",
 "is_evening",
 "is_morning",
 "carrier_delay_rate_smooth",
 "dest_delay_rate_smooth",
 "carrier_hour_delay_rate_smooth",
 "flight_num_delay_rate_smooth",
 "prior_same_day_delay_rate",
 "before_today",
 "avg_delay_rate_for_calendar_day",
 "high_disruption_calendar_day"
)

train_model_df <- train_fe |>
 select(any_of(model_vars)) |>
 filter(!is.na(dep_del15))

test_visible_model_df <- test_visible_fe |>
 select(any_of(model_vars))

test_guess_model_df <- test_guess_fe |>
 select(any_of(model_vars))
```

At this point, `train_model_df` contains the response and engineered predictors for model fitting, while `test_guess_model_df` contains the same predictors for the flights we need to score. The final feature set reflects the main conclusions from the EDA: time of day is a major driver, carrier, destination, route are important, and same-day disruption adds useful information.

Modelling

```
set.seed(100)
n <- nrow(train_model_df)
train_idx <- sample(1:n, size = 0.8 * n)
train_split <- train_model_df[train_idx, ]
test_split <- train_model_df[-train_idx, ]
```

```
compute_auc <- function(auc_df = NULL, train_y, train_prob,
                        test_y, test_prob, model_name){
 auc_train <- roc(train_y, train_prob) |>
 auc()
 auc_test <- roc(test_y, test_prob) |>
 auc()
 accur_train_0.5 <- mean(train_y == (train_prob > 0.5))
 accur_test_0.5 <- mean(train_y == (test_prob > 0.5))
 accur_train_0.6 <- mean(train_y == (train_prob > 0.6))
 accur_test_0.6 <- mean(train_y == (test_prob > 0.6))
 df <- data.frame(model_name, auc_train, auc_test,
                  accur_train_0.5, accur_test_0.5,
                  accur_train_0.6, accur_test_0.6)
 if (is.null(auc_df)){
 return (df)
 } else{
 return (rbind(auc_df, df))
 }
}
```

Logistic Regression

```
# fit model
log_reg <- glm(dep_del15~., family = 'binomial', data = train_split)
# get prob
train_prob <- predict(log_reg, type = "response",
 newdata = train_split)
test_prob <- predict(log_reg, type = "response",
 newdata = test_split)
# get AUC
auc_df <- compute_auc(train_y = train_split$dep_del15,
 train_prob = train_prob,
 test_y = test_split$dep_del15,
 test_prob = test_prob,
 model_name = "Log Reg")
```

LASSO

```
# prep data for glmnet
x_train <- model.matrix(dep_del15 ~ ., train_split)[, -1]
y_train <- train_split$dep_del15
x_test <- model.matrix(dep_del15 ~ ., test_split)[, -1]
y_test <- test_split$dep_del15

cv_lasso <- cv.glmnet(
```

```r
 x_train, y_train,
 family = "binomial",
 alpha = 1
)

train_prob <- predict(cv_lasso, newx = x_train,
 s = "lambda.min", type = "response") |>
 as.vector()

test_prob <- predict(cv_lasso, newx = x_test,
 s = "lambda.min", type = "response") |>
 as.vector()
auc_df <- compute_auc(
 auc_df = auc_df,
 train_y = y_train,
 train_prob = train_prob,
 test_y = y_test,
 test_prob = test_prob,
 model_name = "Log Reg (LASSO)"
)
```

XGBoost

```r
# prep data for xgboost
dtrain <- xgb.DMatrix(data = x_train, label = y_train)
dtest <- xgb.DMatrix(data = x_test, label = y_test)
# xgboost
params <- list(
 objective = "binary:logistic",
 eval_metric = "auc"
)

xgb_model <- xgb.train(
 params = params,
 data = dtrain,
 nrounds = 100,
 verbose = 0
)
train_prob <- predict(xgb_model, dtrain)
test_prob <- predict(xgb_model, dtest)
auc_df <- compute_auc(
 auc_df = auc_df,
 train_y = y_train,
 train_prob = train_prob,
 test_y = y_test,
 test_prob = test_prob,
 model_name = "XGBoost"
)
```

Random Forest

```r
train_split$dep_del15 <- as.factor(train_split$dep_del15)
test_split$dep_del15 <- as.factor(test_split$dep_del15)
```

```r
rf_model <- randomForest(
 dep_del15 ~ .,
 data = train_split,
 ntree = 300, # number of trees
 importance = TRUE
)

train_prob <- predict(rf_model, newdata = train_split, type = "prob")[, "1"]
test_prob <- predict(rf_model, newdata = test_split, type = "prob")[, "1"]

auc_df <- compute_auc(
 auc_df = auc_df,
 train_y = y_train,
 train_prob = train_prob,
 test_y = y_test,
 test_prob = test_prob,
 model_name = "Random Forest"
)
```

Balanced Random Forest

```r
# class weights
class_counts <- table(train_split$dep_del15)

class_weights <- as.numeric(sum(class_counts) / (length(class_counts) * class_counts))
names(class_weights) <- names(class_counts)

brf_model <- randomForest(
 dep_del15 ~ .,
 data = train_split,
 ntree = 300,
 classwt = class_weights
)
train_prob <- predict(brf_model, newdata = train_split, type = "prob")[, "1"]
test_prob <- predict(brf_model, newdata = test_split, type = "prob")[, "1"]
auc_df <- compute_auc(
 auc_df = auc_df,
 train_y = y_train,
 train_prob = train_prob,
 test_y = y_test,
 test_prob = test_prob,
 model_name = "Balanced RF"
)
```

Aggregate All Models

```r
preds_df <- data.frame(
 y_true = as.numeric(as.character(test_split$dep_del15)), # ensure 0/1
 log_reg = predict(log_reg, type = "response", newdata = test_split),
 lasso = predict(cv_lasso, newx = x_test, s = "lambda.min", type = "response") |> as.vector(),
 xgboost = predict(xgb_model, dtest),
 rf = predict(rf_model, newdata = test_split, type = "prob")[, "1"],
```

```r
 brf = predict(brf_model, newdata = test_split, type = "prob")[, "1"]
)
preds_df$avg_prob <- rowMeans(preds_df[, -1])  # exclude y_true
auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = preds_df$y_true,   # using test as both (since ensemble built on test)
  train_prob = preds_df$avg_prob,
  test_y = preds_df$y_true,
  test_prob = preds_df$avg_prob,
  model_name = "Ensemble (Avg Prob)"
)
```

```r
pred_class <- preds_df[, -1] > 0.5  # TRUE/FALSE matrix

# majority vote (at least half the models predict 1)
preds_df$majority_class <- as.numeric(rowMeans(pred_class) > 0.5)
# use proportion of models voting "1" as probability
preds_df$majority_prob <- rowMeans(pred_class)
auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = preds_df$y_true,
  train_prob = preds_df$majority_prob,
  test_y = preds_df$y_true,
  test_prob = preds_df$majority_prob,
  model_name = "Ensemble (Majority Vote)"
)
```

Results - 2025 Visible

```r
compute_vis_auc <- function(auc_df = NULL, y, prob, model_name){
 auc <- roc(y, prob) |>
 auc()
 accuracy <- mean(y == (prob > 0.5), na.rm = T)
 df <- data.frame(model_name, auc, accuracy)
 if (is.null(auc_df)){
 return (df)
 } else{
 return (rbind(auc_df, df))
 }
}
```

```r
test_visible_model_df <- test_visible_model_df |>
  filter(!is.na(dep_del15))
y_true = as.numeric(as.character(test_visible_model_df$dep_del15))
# log reg
log_reg_prob = predict(log_reg, type = "response",
                  newdata = test_visible_model_df)
vis_auc <- compute_vis_auc(y = y_true, prob = log_reg_prob,
                           model_name = "Log Reg")
# lasso
x_vis_test <- model.matrix(dep_del15 ~ ., test_visible_model_df)[, -1]
y_vis <- test_visible_model_df$dep_del15
lasso = predict(cv_lasso, newx = x_vis_test, s = "lambda.min", type = "response") |> as.vector()
```

```r
vis_auc <- vis_auc |>
  compute_vis_auc(y = y_true, prob = lasso,
                          model_name = "LASSO")
# xgboost
dtest <- xgb.DMatrix(data = x_vis_test, label = y_vis)
xgboost = predict(xgb_model, dtest)
vis_auc <- vis_auc |>
  compute_vis_auc(y = y_true, prob = xgboost,
                          model_name = "XGBoost")
# RF
rf = predict(rf_model, newdata = test_visible_model_df, type = "prob")[, "1"]
vis_auc <- vis_auc |>
  compute_vis_auc(y = y_true, prob = rf,
                          model_name = "RF")
# Balanced RF
brf = predict(brf_model, newdata = test_visible_model_df, type = "prob")[, "1"]
vis_auc <- vis_auc |>
  compute_vis_auc(y = y_true, prob = brf,
                          model_name = "Balanced RF")
# Avg Prov
avg_prob <- data.frame(log_reg_prob, lasso, xgboost, rf, brf) |>
  rowMeans()
vis_auc <- vis_auc |>
  compute_vis_auc(y = y_true, prob = avg_prob,
                          model_name = "Avg Prob")
```

```r
vis_auc |>
  arrange(desc(auc))
```

```
##    model_name       auc  accuracy
## 1     Log Reg 0.6732118 0.8851216
## 2       LASSO 0.6730402 0.8850163
## 3          RF 0.6695103 0.8820680
## 4    Avg Prob 0.6687152 0.8843845
## 5 Balanced RF 0.6353019 0.8858587
## 6     XGBoost 0.6216265 0.8806992
```

```r
vis_auc |>
  arrange(desc(accuracy))
```

```
##    model_name       auc  accuracy
## 1 Balanced RF 0.6353019 0.8858587
## 2     Log Reg 0.6732118 0.8851216
## 3       LASSO 0.6730402 0.8850163
## 4    Avg Prob 0.6687152 0.8843845
## 5          RF 0.6695103 0.8820680
## 6     XGBoost 0.6216265 0.8806992
```